

2024 SCHEME

KTUNOTES.LIVE

SCAN THE QR CODES

join our WhatsApp group



For WhatsApp group

get into our website



website for more materials

CLICK THE LOGO TO JOIN



OUR TELEGRAM CHANNEL

for notes and discussion and updates



OUR WHATSAPP GROUP

for note updates and notifications



OUR WEBSITE

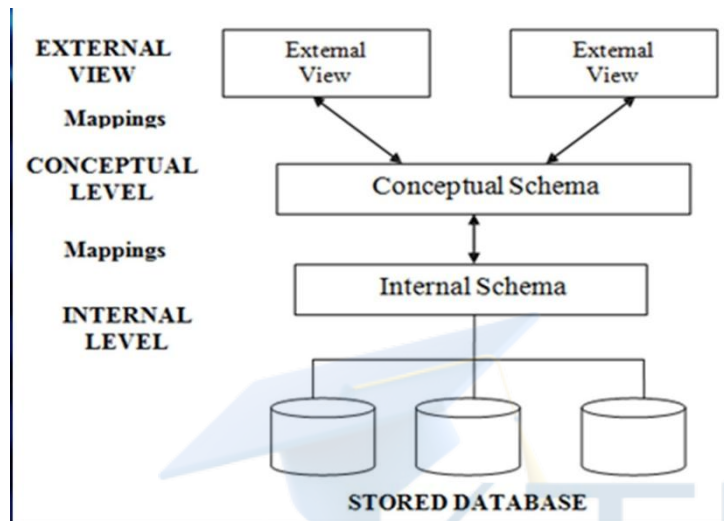
get in and download all 2024 materials

PART A

1. Explain the Three-Schema Architecture of a DBMS with a neat diagram.

How does it provide logical and physical data independence?

Three-Schema Architecture of DBMS



1. External Level (View Level)

- Highest level
- Represents how users view the data
- Each user can have a different view
- Hides rest of the database

2. Conceptual Level (Logical Level)

- Describes the overall structure of the database
- Includes entities, relationships, constraints
- Independent of physical storage

3. Internal Level (Physical Level)

- Lowest level
- Describes how data is stored physically
- Includes file structures, indexing, storage

Data Independence

1. Logical Data Independence

- Ability to change **conceptual schema** without affecting external views
- Users don't need to change their applications

☞ Example: Add a new column (DOB) → User views remain same

2. Physical Data Independence

- Ability to change **internal schema** without affecting conceptual schema
- Storage changes won't affect logical structure

☞ Example: Change indexing or file structure

2. Distinguish between Super key, Candidate key, and Primary key. Illustrate with a real-world example from a university student registration system.

Feature	Super Key	Candidate Key	Primary Key
Definition	Unique identifier (may have extra attributes)	Minimal super key	Selected candidate key
Redundancy	Allowed	Not allowed	Not allowed
Count	Many	Few	Only one
Example	{ID, Name}	{ID}	{ID}

Real-world Example: University Student Registration System

Consider a STUDENT relation (table) with the following attributes:

STUDENT (StudentID, Name, Email, SocialSecurityNumber, PhoneNumber, Address)

Let's assume the following:

- StudentID is unique for each student.
- Email is unique for each student.
- SocialSecurityNumber (SSN) is unique for each student.
- Name, PhoneNumber, and Address are not necessarily unique on their own.

Step 1: Identify Super Keys

Possible super keys:

- {Student_ID}
- {Aadhaar_No}
- {Email}
- {Student_ID, Name}
- {Email, Phone}
- {Student_ID, Email, Name}

Step 2: Candidate Keys

Minimal ones (remove extra attributes):

- {Student_ID}
- {Aadhaar_No}
- {Email}

Step 3: Primary Key

Choose one:

☞ **Primary Key = Student_ID**

3. Write an SQL trigger on an Employees table that automatically logs the old and new salary values into an Audit_Log table whenever an employee's salary is updated Give an example.

An SQL trigger is a special type of stored procedure that automatically executes when a specified event (INSERT, UPDATE, or DELETE) occurs on a table.

Scenario:

- **Employees table: (EmployeeID, Name, Salary)**
- **Audit_Log table: (LogID, EmployeeID, OldSalary, NewSalary, ChangeDate)**

```
CREATE TRIGGER salary_update_trigger
```

```
AFTER UPDATE ON Employees
```

```
FOR EACH ROW
```

```
WHEN (OLD.Salary <> NEW.Salary)
```

```
BEGIN
```

```
    INSERT INTO Audit_Log (Log_ID, Emp_ID, Old_Salary, New_Salary, Change_Date)  
VALUES (Audit_seq.NEXTVAL, OLD.Emp_ID, OLD.Salary, NEW.Salary, SYSDATE);
```


END;

Example:

Employees Table

Emp_ID	Name	Salary
101	Riya	30000

Update Query: UPDATE Employees SET Salary = 40000 WHERE Emp_ID = 101;

After Update: Employee Table

Emp_ID	Name	Salary
101	Riya	40000

Audit_Log Table

Log_ID	Emp_ID	Old_Salary	New_Salary	Change_Date
1	101	30000	40000	(current date)

The Audit_Log table automatically recorded the salary change for EmployeeID 101.

4. Explain the ER-to-Relational mapping rule for a Binary M: N Relationship Type. Illustrate with an example showing the resulting relational schema.

Rule (Binary M: N Relationship)

For a binary many-to-many (M: N) relationship between two entities:

☞ Create a new relation (table) for the relationship

☞ Include:

- Primary keys of both entities as foreign keys
- These together form a composite primary key
- Add any relationship attributes

Example: Student – Course System

ER Concept

- Entity 1: **STUDENT**(Student_ID, Name)
- Entity 2: **COURSE**(Course_ID, Course_Name)
- Relationship: **ENROLLS** (Many students ↔ many courses)
- Relationship Attribute: **Grade**

Mapping to Relational Schema

Step 1: Create Entity Tables

STUDENT(Student_ID PRIMARY KEY, Name)

COURSE(Course_ID PRIMARY KEY, Course_Name)

Step 2: Create Relationship Table

ENROLLS(
Student_ID,
Course_ID,
Grade,
PRIMARY KEY (Student_ID, Course_ID),
FOREIGN KEY (Student_ID) REFERENCES STUDENT(Student_ID),
FOREIGN KEY (Course_ID) REFERENCES COURSE(Course_ID)
)

Table	Description
STUDENT	Stores student details
COURSE	Stores course details
ENROLLS	Links students and courses

5. Explain the anomalies (insertion, deletion, update) that may occur in an unnormalized relation, with a suitable example.

Anomalies are problems that arise in unnormalized (or poorly normalized) relational database designs, particularly when a single table stores data that belongs to multiple entities or has transitive dependencies. These anomalies lead to data redundancy, inconsistency, and integrity issues.

Consider:

STUDENT_COURSE(Student_ID, Student_Name, Course_ID, Course_Name, Instructor)

Student_ID	Student_Name	Course_ID	Course_Name	Instructor
1	Riya	C1	DBMS	Anil
1	Riya	C2	Java	Binu
2	Arun	C1	DBMS	Anil

Course and Instructor data are **repeated**.

1. Insertion Anomaly

Cannot insert some data without other data

☞ Example:

- Want to add a new course (C3, AI, Instructor: John)
- But no student enrolled yet → Cannot insert

2. Deletion Anomaly

Deleting a record may **lose important information**

☞ Example:

- Delete student Arun (only student in DBMS)
- DBMS course info also lost ✗

3. Update Anomaly

Updating data requires **multiple changes**

☞ Example:

- Instructor “Anil” changes to “Anil Kumar”
- Must update in **all rows**
- Missing one → inconsistency ✗

6. Explain the Two-Phase Locking (2PL) protocol. Describe the growing and shrinking phases, and explain why 2PL guarantees Serializability.

The Two-Phase Locking (2PL) protocol is a concurrency control mechanism used in database management systems to ensure Serializability of transactions. Serializability means that the concurrent execution of multiple transactions produces the same result as some serial execution of those same transactions.

Dividing a transaction into **two phases**:

1. Growing Phase
2. Shrinking Phase

1. Growing Phase

- In this phase, a transaction can acquire locks on data items, but it cannot release any locks.
- The transaction gradually acquires all the locks it needs. It can request shared (read) locks or exclusive (write) locks. The number of locks held by the transaction can only increase during this phase.

3. Shrinking Phase

- In this phase, a transaction can release locks, but it cannot acquire any new locks.
- After acquiring all necessary locks and completing its critical operations, the transaction begins to release its locks. The number of locks held by the transaction can only decrease during this phase.

Once a transaction releases a lock → it **cannot request any new lock**.

Example

Consider two transactions:

T1: Lock-X(A) → Read(A) → Write(A) → Unlock(A)

T2: Lock-S(A) → Read(A) → Unlock(A)

2PL Guarantees Serializability

2PL guarantees Serializability because it prevents inconsistencies that arise from conflicting operations (read-write, write-read, write-write) by ensuring that transactions maintain their required locks.

- **Prevents Cascading Rollbacks (2PL / Strict 2PL)**
Transactions do not expose uncommitted data → avoids chain rollbacks.
- **Strict 2PL (Important in practice)**
Holds **write (exclusive) locks until commit/abort** → prevents **dirty reads**.
- **Prevents Lost Updates**
Exclusive locks ensure only one transaction updates a data item at a time.
- **Prevents Unrepeatable Reads**
Shared locks ensure data cannot change while a transaction is reading it multiple times.
- **Prevents Phantom Reads (Advanced cases)**
With proper locking (e.g., range/predicate locks), insertion of new matching rows is prevented.
- **Ensures Serializability**
Locking order (lock point) defines a **serial execution order equivalent**.

7. Explain the CAP theorem. For each pair of two properties (CA, CP, AP), name one real-world database system that prioritizes that combination and briefly justify.

The **CAP theorem** (Consistency, Availability, Partition Tolerance) is a fundamental principle in distributed computing, particularly relevant to distributed database systems. It states that it is impossible for a distributed data store to simultaneously provide all three of the following guarantees:

- **Consistency (C):** All nodes in the system see the same data at the same time. After a write operation, any subsequent read operation must return the latest data, or an error.
- **Availability (A):** Every request receives a response, without guarantee that it's the latest write. The system remains operational and responsive to requests, even if some nodes fail.
- **Partition Tolerance (P):** The system continues to operate despite arbitrary failures of network links, causing a "partition" where some nodes cannot communicate with others.
-

The Three Combinations

1. CA (Consistency + Availability)

☞ No partition tolerance (ideal system, no network failures)

Example: MySQL (Single-node setup)

- Ensures **strong consistency** (ACID transactions)
- Always available (in absence of partition)
- Not designed to handle network partition

2. CP (Consistency + Partition Tolerance)

☞ May sacrifice availability during failures

Example: Apache HBase

- Ensures **consistent data across nodes**
- Handles network partitions
- May **reject requests (not available)** during partition

3. AP (Availability + Partition Tolerance)

☞ May sacrifice consistency

Example: Apache Cassandra

- Always **responds to requests**
- Works even during partitions
- Data may be **temporarily inconsistent** (eventual consistency)

8. Compare Key-Value Stores and Document Stores in terms of data model, query flexibility, and a practical use case for each.

Feature	Key-Value Stores	Document Stores
Data Model	Key -> Opaque Value (string, JSON, blob)	Key -> Self-describing Document (JSON, BSON, XML)
Schema	Schema-less (value content unknown to DB)	Flexible schema (schema-less, but documents have internal structure)
Query Flex.	Only by Key	By Key, by field content, nested fields, ranges, etc.
Data Complexity	Simple, flat data; opaque to DB	Semi-structured, hierarchical data; interpreted by DB
Use Case	Caching, session management, simple lookups	Content management, user profiles, product catalogs
Example DB	Redis, Riak	MongoDB, Couchbase

PART B

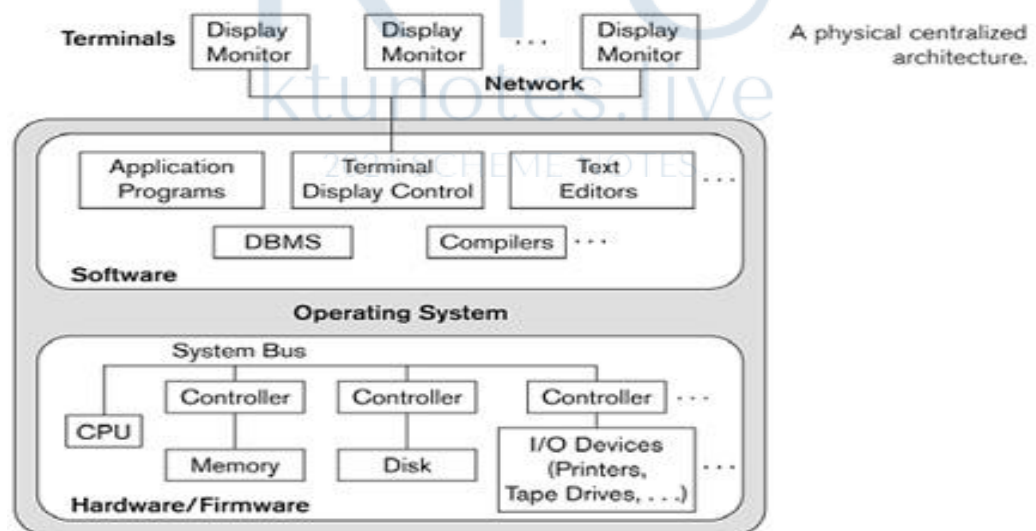
Module 1

- 9 (a) Explain centralized and client/server DBMS architectures with diagrams. Compare their advantages and limitations in the context of a modern enterprise application.

Database Management System (DBMS) architectures describe how the various components of a database system are distributed and interact. Historically, centralized systems were dominant, but with the rise of networking and distributed computing, client/server models became prevalent, evolving further into multi-tiered architectures.

1. Centralized DBMS Architecture

In a centralized DBMS architecture, all the components of the DBMS, the database itself, and the application programs that access the database reside on a single computer system (typically a mainframe or a powerful server). Users interact with the system through terminals (dumb terminals) that do not perform any processing logic; they simply send input to the central system and display output received from it. All data processing, application logic, and database management occur on the central machine.



Advantages in a Modern Enterprise Context:

- **Simplicity and Control:** Easier to set up, manage, and administer as everything is on one machine. Centralized control over data and processes.
- **Data Consistency:** Easier to maintain data consistency and integrity because there's only one copy of the data and all transactions are processed in a single environment.

- **Security:** Security management is simpler as it's concentrated at a single point.
- **Lower Network Traffic:** Interaction with the database happens locally on the server, minimizing network latency for database operations (though terminal I/O still uses the network).

Limitations in a Modern Enterprise Context:

- **Scalability Bottleneck:** The single server can become a performance bottleneck. Scaling up (adding more resources to the single machine) has physical and cost limits.
- **Single Point of Failure:** If the central server fails, the entire system becomes unavailable, leading to significant downtime.
- **Limited Accessibility:** Access is typically restricted to terminals directly connected or through specific protocols, which might not be suitable for diverse client types (web browsers, mobile apps).
- **High Initial Cost:** Mainframe systems or very powerful servers can have a high initial acquisition and maintenance cost.
- **Poor for Distributed Data:** Not suitable for scenarios where data naturally resides in different geographical locations.
- **Not Cloud-Native:** This architecture doesn't align with modern cloud computing principles of distributed, elastic, and fault-tolerant systems.

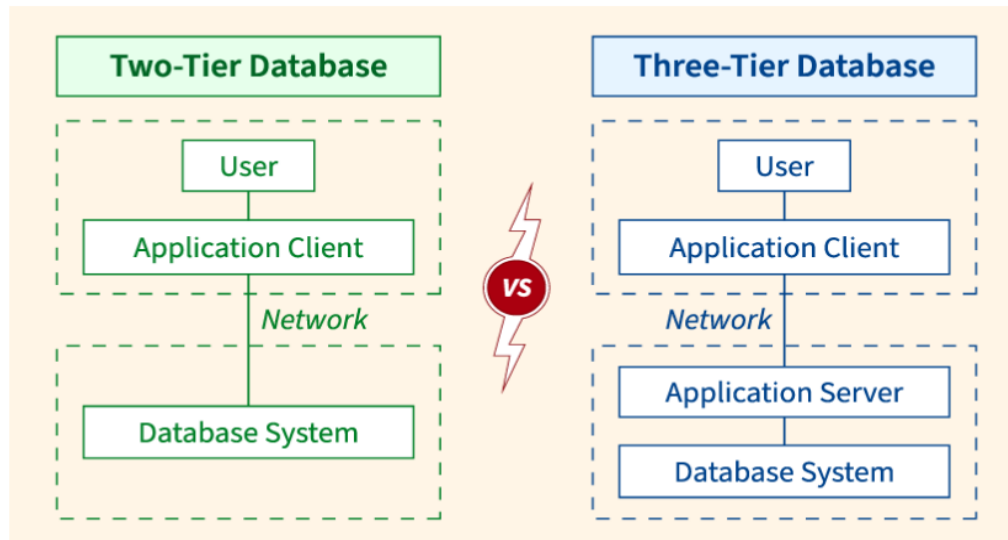
2. Client/Server DBMS Architecture

In a client/server DBMS architecture, the processing workload is distributed between two main types of components: clients and servers.

Client (Front-end): Typically, a workstation, PC, or mobile device that runs the user interface and application logic. It sends requests to the database server.

Server (Back-end): A more powerful machine that runs the DBMS software and manages the database. It processes client requests, executes database operations, and sends results back to the client.

This is a two-tier architecture, where the client handles presentation and some application logic, and the server handles database logic. More complex architectures can involve three or more tiers (e.g., separating application logic onto its own server, forming a "middle tier").



Advantages in a Modern Enterprise Context:

- **Scalability:** Easier to scale compared to centralized systems. You can add more client machines or upgrade the server (vertical scaling) or distribute the database across multiple servers (horizontal scaling, though this leads to distributed DBMS).
- **Cost-Effectiveness:** Often uses off-the-shelf client hardware (PCs), reducing overall hardware costs compared to mainframes.
- **Flexibility and Accessibility:** Supports a wider range of client applications (desktop apps, web apps, mobile apps). Users can access the database from various locations.
- **Specialization:** Allows for specialization of tasks. Clients handle user interface and some business logic, while servers optimize for data storage and retrieval.
- **Improved Performance:** By distributing the workload, the system can handle more users and larger transaction volumes. Database processing happens on a powerful server, freeing up client resources.
- **Enhanced Security:** Database security can be managed centrally on the server, and client access can be restricted.

Limitations in a Modern Enterprise Context:

- **Increased Complexity:** More complex to design, implement, and administer than a centralized system due to distributed components and network communication.
- **Network Dependence:** Performance is highly dependent on network bandwidth and latency. High network traffic can become a bottleneck.

- **Security Concerns:** Securing communication between clients and servers over a network requires careful implementation.
- **Load Balancing:** Managing client requests and distributing the load efficiently across servers (especially in multi-tier or distributed setups) can be challenging.
- **Development Complexity:** Developing client-side applications that effectively interact with the database server requires more sophisticated programming.

Comparison and Context in a Modern Enterprise Application:

In the context of a **modern enterprise application**, the **client/server DBMS architecture (and its multi-tier extensions)** is overwhelmingly preferred and almost universally adopted, while the centralized model is largely obsolete for new development

9 (b). What are weak entity types? Explain with an example from the COMPANY database. Show how weak entities and their identifying relationships are represented in an ER diagram.

Feature	Centralized DBMS Architecture	Client/Server DBMS Architecture
Processing Power	All on one powerful machine	Distributed between client and server
Scalability	Limited (vertical scaling only, expensive)	Highly scalable (vertical and horizontal)
Fault Tolerance	Single point of failure	Better (server failure impacts many, but clients can be diverse; can have failovers)
Accessibility	Limited (terminals)	Wide (desktop, web, mobile clients)
Network Traffic	Low for DB ops, high for terminal I/O	High for client-server communication
Development Cost	Potentially high (mainframe)	Generally lower (commodity hardware, diverse client dev)
Complexity	Simpler to manage at a basic level	More complex to design, implement, and secure
Modern Relevance	Niche (legacy systems), largely obsolete	Standard for virtually all modern applications

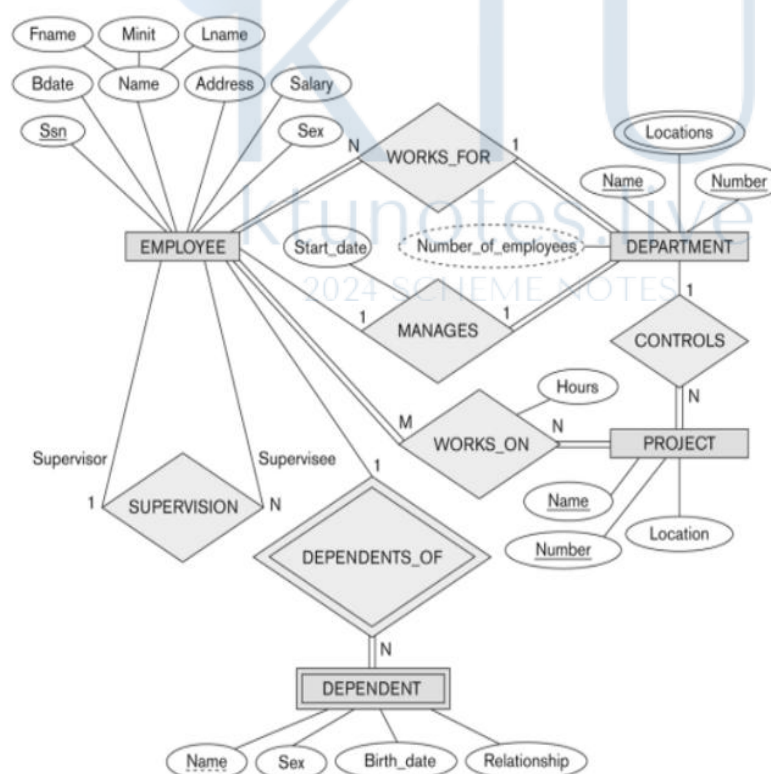
In the Entity-Relationship (ER) model, an **entity type** represents a collection of entities that share common characteristics. Entity types can be classified as either **strong (regular)** or **weak**.

A **weak entity type** is an entity type that does not have its own primary key attributes. Instead, its existence and identification are dependent on another entity type, called the **identifying (or owner) entity type**. Without the existence of an owner entity, a weak entity cannot exist or be uniquely identified.

Key Characteristics of Weak Entity Types:

1. **Existence Dependence:** A weak entity cannot exist without its owner entity. If the owner entity is deleted, all its associated weak entities must also be deleted.
2. **Identification Dependence:** A weak entity does not have enough attributes to form a primary key on its own. It derives its primary key (or at least part of it) from the primary key of its owner entity.
3. **Identifying Relationship:** The relationship between a weak entity type and its owner entity type is called an **identifying relationship**. This relationship is always a one-to-many (1:N) relationship from the owner to the weak entity. Each weak entity is related to *exactly one* owner entity.

Example from the COMPANY Database



How Weak Entities and their Identifying Relationships are Represented in an ER Diagram

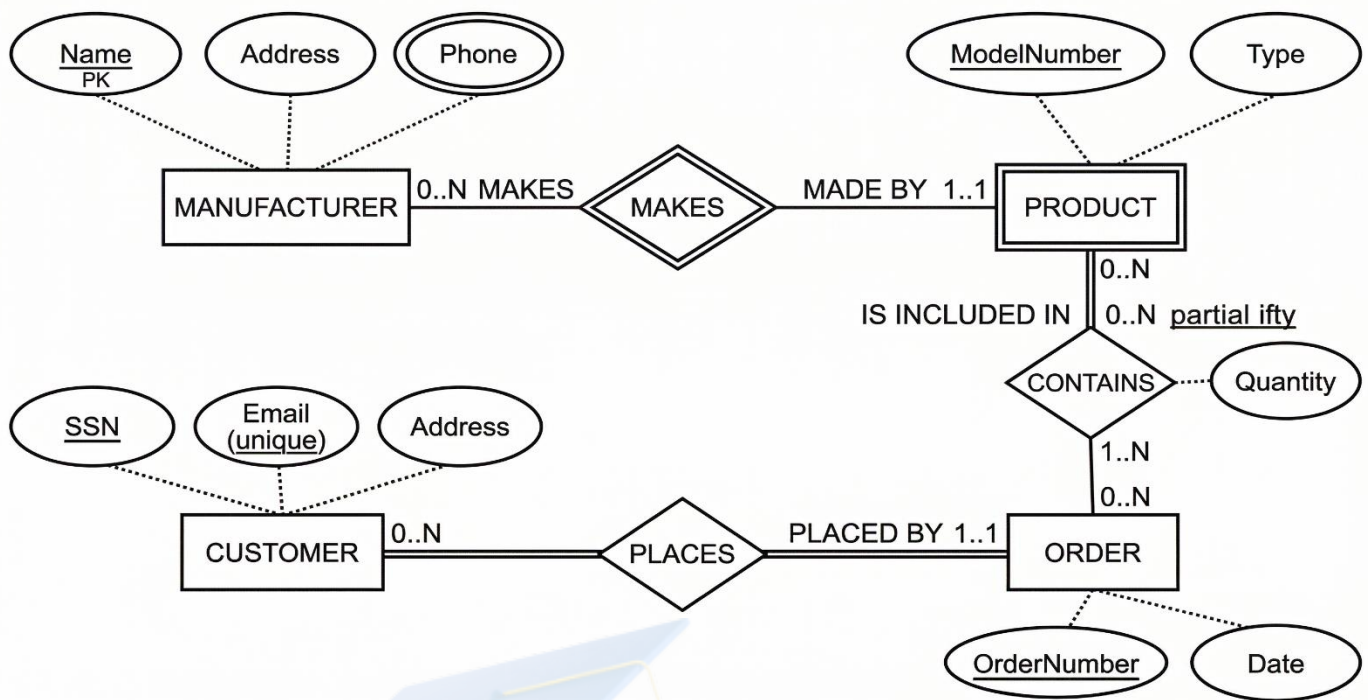
In an ER diagram, weak entities and their identifying relationships have specific notations:

1. **Weak Entity Type:** Represented by a **double-lined rectangle**.
2. **Identifying Relationship Type:** Represented by a **double-lined diamond**.
3. **Partial Key (Discriminator):** The attribute(s) that uniquely identify a weak entity *relative to its owner entity* are shown with a **dashed underline**.
4. **Connecting Lines:** The weak entity type is connected to the identifying relationship, which in turn is connected to the owner entity type. The connecting line from the owner entity to the identifying relationship typically shows a **1: N cardinality**, with the weak entity being on the 'many' side. There is also a total participation constraint from the weak entity to the identifying relationship (double line from weak entity to relationship), indicating that every weak entity instance *must* participate in the identifying relationship

10. Draw an ER diagram for an Online Retail System based on the following requirements:

- **Manufacturers have a unique name, address, and phone number.**
- **Products have a model number and a type. Each product is made by one manufacturer. Different manufacturers may have different products with the same model number, but no manufacturer has two products with the same model number.**
- **Customers are identified by a unique social security number. They have email addresses and physical addresses. Several customers may live at the same address, but no two customers have the same email.**
- **An order has a unique order number and a date. An order is placed by one customer. Each order contains one or more products, with a quantity for each product.**

Clearly identify all entities, relationships, attributes, keys, cardinality ratios, and participation constraints.



1. Entities and Attributes

- Manufacturer: * Attributes: Name (Primary Key), Address, Phone Number.
- Product (Weak Entity): * Since different manufacturers can use the same model number, a Product is a weak entity that depends on the Manufacturer.
 - Attributes: Model Number (Partial Key), Type.
- Customer: * Attributes: SSN (Primary Key), Email (Unique), Address.
- Order: * Attributes: Order Number (Primary Key), Date.

2. Relationships and Cardinality

The connections between these entities define how data flows through the system:

Relationship	Entities Involved	Cardinality	Participation
Makes	Manufacturer & Product	1:N	One manufacturer makes many products; a product is made by exactly one manufacturer.

Relationship	Entities Involved	Cardinality	Participation
Places	Customer & Order	1:N	One customer can place many orders; an order belongs to exactly one customer.
Contains	Order & Product	M:N	An order contains one or more products; a product can appear in many different orders.

3. Key Constraints & Logic

- **Weak Entity Logic:** The **Product** entity is represented with a double rectangle because its identity is tied to the Manufacturer. Its unique identification is a combination of the Manufacturer's Name and its own Model Number.
- **Descriptive Attribute:** The relationship **Contains** has its own attribute: **Quantity**. This is because quantity belongs to the specific instance of a product being inside a specific order, rather than the product or order alone.
- **Participation: * Total Participation (Double Line):** Every **Order** must be placed by a Customer, and every **Order** must contain at least one Product.
 - **Partial Participation (Single Line):** A **Manufacturer** might exist in the database without having any products listed yet.

Module 2 ME NOTES

11(a). Consider the following schema:

Suppliers (SID, sname, address)

Parts (PID, pname, colour)

Catalog (SID, PID, price)

Catalog[SID] references Suppliers[SID]; Catalog[PID] references Parts[PID]

Write relational algebra expressions for:

- Find all prices for parts that are red or green.
- Find the SIDs of all suppliers who supply a red part with price greater than 100.
- Find the names of suppliers who supply every red part (use division).

i. Find all prices for parts that are red or green.

This query requires us to first identify the PIDs of parts that are either 'red' or 'green' from the Parts relation. Then, we need to join these PIDs with the Catalog relation to get the corresponding prices and finally project the price attribute.

Select PIDs where colour is 'red' or 'green' from Parts:

$$\pi_{PID}(\sigma_{colour='red' \vee colour='green'}(Parts))$$

Join this result with Catalog on PID:

$$(\pi_{PID}(\sigma_{colour='red' \vee colour='green'}(Parts))) \bowtie_{PID} Catalog$$

Project the price attribute:

$$\pi_{price}((\pi_{PID}(\sigma_{colour='red' \vee colour='green'}(Parts))) \bowtie_{PID} Catalog)$$

Combined Expression:

$$\pi_{price}((\sigma_{colour='red' \vee colour='green'}(Parts)) \bowtie_{Parts.ID=Catalog.PID} Catalog)$$

ii. Find the SIDs of all suppliers who supply a red part with price greater than 100.

This query involves filtering parts by colour and price, and then joining with Catalog to find the SIDs of suppliers who meet these criteria.

Select PIDs where colour is 'red' from Parts:

$$\rho_{red} Parts(\sigma_{colour='red'}(Parts)) \text{ (Renaming for clarity, not strictly necessary)}$$

Let's just use the selection directly for now.

Select entries from Catalog where price > 100:

$$\sigma_{price>100}(Catalog)$$

Join the Parts relation (filtered by 'red') with the Catalog relation (filtered by price > 100) on PID.

$$(\sigma_{colour='red'}(Parts)) \bowtie_{Parts.ID=Catalog.PID} (\sigma_{price>100}(Catalog))$$

From this result, project the SID:

$\pi_{SID}((\sigma_{colour='red'}(Parts)) \bowtie Parts.ID=Catalog.PID(\sigma_{price>100}(Catalog)))$

iii. Find the names of suppliers who supply every red part (use division).

This query requires finding suppliers who supply *all* red parts. This is a classic application of relational division.

- Get all **red parts (PID)**
- Get (SID, PID) pairs from Catalog
- Use **division** to find suppliers who supply *all red parts*
- Join with Suppliers to get name

$RedParts \leftarrow \pi_{PID}(\sigma_{colour='red'}(Parts))$

$SupplierParts \leftarrow \pi_{SID, PID}(Catalog)$

$Result \leftarrow SupplierParts \div RedParts$

$Final \leftarrow \pi_{sname}(Result \bowtie Suppliers)$

9(b). Explain the concept of Views in SQL. Write SQL statements to create a view that shows the supplier name and total number of parts supplied. Discuss one advantage and one security benefit of using views.

A View in SQL is essentially a virtual table based on the result-set of an SQL statement. Unlike a regular table, a view does not store the data itself; instead, it stores the query and dynamically retrieves the data from the underlying "base tables" whenever you access it.

Creating the View

To show the supplier name and the total number of parts they supply, we need to join the Suppliers and Catalog tables and use the COUNT aggregate function.

CREATE VIEW SupplierPartCount AS SELECT S. sname, COUNT(C.PID) AS TotalParts FROM Suppliers S JOIN Catalog C ON S.SID = C.SID GROUP BY S. sname;

Once created, you can query this view just like a table: `SELECT * FROM SupplierPartCount;`

Benefits of Using Views

1. Advantage: Simplification of Complex Queries

- Views allow you to hide the complexity of the underlying database schema. Instead of writing a long query with multiple joins and aggregations (like the one above) every time you need that information, you can simply call the view.
- This makes the database much easier to interact with for end-users or application developers.

2. Security Benefit: Data Abstraction and Column-Level Security

- Views are a powerful tool for implementing the Principle of Least Privilege. You can grant a user permission to access a view without giving them access to the underlying tables.
- Example: In an "Employees" table, you might want a manager to see names and department IDs but hide sensitive information like Social Security Numbers or salaries. By creating a view that excludes those sensitive columns and granting the manager access *only* to the view, the sensitive data remains protected.

12. Consider the following database schema:

Employees (EmployeeID INT PRIMARY KEY, FirstName VARCHAR (50), LastName VARCHAR (50), JobTitle VARCHAR (100), Salary DECIMAL(10,2), HireDate DATE, DepartmentID INT)

Departments (DepartmentID INT PRIMARY KEY, DepartmentName VARCHAR (100), ManagerID INT)

Foreign Key: Employees.DepartmentID references Departments.DepartmentID

Write SQL queries for the following:

- Create both tables with appropriate constraints, including a CHECK constraint that Salary > 0.**
- Calculate the average salary per department and display only departments where the average salary exceeds 50,000.**
- List the employee name and department name for the employee with the highest salary in each department using a subquery.**
- Create an assertion that ensures no department has more than 100 employees.**

v. Find employees hired in the last 2 years who earn more than the average salary of their department.

(i) Create both tables with appropriate constraints, including a CHECK constraint that Salary > 0.

- Create Departments table first so Employees can reference it

```
CREATE TABLE Departments (DepartmentID INT PRIMARY KEY,
DepartmentName VARCHAR (100) NOT NULL, ManagerID INT);
```

- Create Employees table: Constraint: Salary must be positive

```
CREATE TABLE Employees (EmployeeID INT PRIMARY KEY, FirstName
VARCHAR (50) NOT NULL, LastName VARCHAR (50) NOT NULL, JobTitle
VARCHAR (100), Salary DECIMAL (10, 2) CHECK (Salary > 0), HireDate DATE,
DepartmentID INT, FOREIGN KEY (DepartmentID) REFERENCES
Departments(DepartmentID));
```

(ii). Average Salary per Department (HAVING clause)

To filter based on an aggregate result (like AVG), we must use the HAVING clause rather than WHERE.

```
SELECT DepartmentID, AVG(Salary) AS AverageSalary FROM Employees GROUP BY
DepartmentID HAVING AVG(Salary) > 50000;
```

iii. Highest Salary per Department (Correlated Subquery)

This query identifies the top earner in every department by comparing each employee's salary to the maximum salary within their specific DepartmentID.

```
SELECT E.FirstName, E.LastName, D.DepartmentName, E.Salary
```

```
FROM Employees E
```

```
JOIN Departments D ON E.DepartmentID = D.DepartmentID
```

```
WHERE E.Salary = (
```

```
SELECT MAX(Salary)
```

```
FROM Employees
```

```
WHERE DepartmentID = E.DepartmentID
```

```
);
```

iv. Database Assertion

An ASSERTION is a check that spans multiple rows or tables. Note that while part of the SQL standard, many popular RDBMS (like MySQL or PostgreSQL) do not support CREATE ASSERTION directly and instead use **Triggers** or **Check Constraints** on specific columns.

```
CREATE ASSERTION LimitEmployeesPerDept
```

```
CHECK (
```

```
NOT EXISTS (
```

```
SELECT DepartmentID
```

```
FROM Employees
```

```
GROUP BY DepartmentID
```

```
HAVING COUNT(EmployeeID) > 100
```

```
));
```

v. Recent Hires Earning Above Department Average

This query combines date filtering with a subquery to compare individual salaries against the departmental mean.

```
SELECT FirstName, LastName, Salary, DepartmentID
```

```
FROM Employees E
```

```
WHERE HireDate >= DATEADD (year, -2, GETDATE ())
```

```
AND Salary > (SELECT AVG(Salary) FROM Employees
```

```
WHERE DepartmentID = E.DepartmentID);
```

```
-- Syntax may vary slightly (e.g., CURRENT_DATE - INTERVAL '2 years')
```

Module 3

13(a). Explain functional dependency with an example. How do functional

dependencies help in identifying redundancy and guiding normalization?

1. The Concept of Functional Dependency

A functional dependency occurs when one attribute (or a set of attributes) uniquely determines the value of another attribute. We denote this as:

$$X \rightarrow Y$$

This reads as "**X functionally determines Y**" or "**Y is functionally dependent on X.**" This means that if two rows have the same value for X, they *must* have the same value for Y.

EmployeeID (X)	FirstName	Department (Y)	Dept_Head
101	Alice	IT	Steve
102	Bob	HR	Sarah
101	Alice	IT	Steve

- **EmployeeID → FirstName:** If I know the EmployeeID is 101, the name will always be Alice.
- **Department → Dept_Head:** If the department is "IT", the head will always be "Steve".

2. Identifying Redundancy

Functional dependencies reveal **Redundancy** when non-key attributes determine other non-key attributes.

In the example above, notice the dependency **Department → Dept_Head**. If we have 1,000 employees in the IT department, we will store the name "Steve" 1,000 times.

- **The Issue:** This is redundant data. If Steve leaves and "Monica" becomes the head, we have to update 1,000 rows (**Update Anomaly**). If we delete all IT employees, we lose the information of who the IT head was (**Deletion Anomaly**).

3. Guiding Normalization

Normalization is the process of organizing data to minimize redundancy. Functional dependencies act as the "GPS" for this process:

- **First Normal Form (1NF):** Ensures attributes are atomic (no sets or lists).
- **Second Normal Form (2NF):** Uses FDs to eliminate **Partial Dependencies**. (A non-key attribute depends on only *part* of a composite primary key).
- **Third Normal Form (3NF):** Uses FDs to eliminate **Transitive Dependencies**. This happens when $A \rightarrow B$ and $B \rightarrow C$; therefore, $A \rightarrow C$.
 - In our example: $\text{EmployeeID} \rightarrow \text{Department}$ and $\text{Department} \rightarrow \text{Dept_Head}$. This is a transitive dependency that 3NF would fix by moving Department details to a separate table.

13(b). A relation R (A, B, C, D, E) has the following functional dependencies: $A \rightarrow B$, $BC \rightarrow D$, $D \rightarrow E$, $A \rightarrow C$

- Identify the candidate key(s).
- Determine whether R is in 1NF, 2NF, and 3NF. Give reasons for each.
- If R is not in 3NF, decompose it into 3NF relations.

FDs:

1. $A \rightarrow B$
2. $BC \rightarrow D$
3. $D \rightarrow E$
4. $A \rightarrow C$

i. Identify the Candidate Key(s)

To find the candidate key, we look for an attribute whose "closure" includes all other attributes in the relation. A^+ (Closure of A):

- From A , we get A (Self).
- From $A \rightarrow B$, we get B .
- From $A \rightarrow C$, we get C .
- Now we have $\{A, B, C\}$. From $BC \rightarrow D$, we get D .

- Now we have $\{A, B, C, D\}$. From $D \rightarrow E$, we get E .
- $A^+ = \{A, B, C, D, E\}$.
- Since the closure of A contains all attributes of the relation, **A is the Candidate Key.**

ii. Determine Normal Forms (1NF, 2NF, 3NF)

1NF (First Normal Form)

- Status: Yes.
- Reason: We assume all attributes are atomic (no multi-valued attributes like lists or sets).

2NF (Second Normal Form)

- Status: Yes.
- Reason: 2NF requires that there are no Partial Dependencies (where a non-prime attribute depends on only part of a candidate key). Since our candidate key is a single attribute (A), any dependency on the key is a "full" dependency. There is no "part" of A to depend on.

3NF (Third Normal Form)

- Status: No.
- Reason: 3NF requires that there are no Transitive Dependencies. A transitive dependency occurs when a non-prime attribute determines another non-prime attribute ($\text{Key} \rightarrow X \rightarrow Y$).
 - In our case: $A \rightarrow D$ (derived) and $D \rightarrow E$.
 - Since D is not a candidate key and E is not a prime attribute, the FD $D \rightarrow E$ is a transitive dependency. This violates 3NF.

iii. Decompose into 3NF Relations

To reach 3NF, we must eliminate transitive dependencies by splitting the relation. We follow the FDs to create new tables where every determinant is a candidate key.

Step 1: Handle $D \rightarrow E$

Create a relation for this dependency.

- **R1(D, E)**— Here, D is the Primary Key.

Step 2: Handle $BC \rightarrow D$

Since B and C are determined by A, we look at the chain. However, $BC \rightarrow D$ is its own functional unit.

- **R2(B, C, D)** — Here, (B, C) is the Primary Key.

Step 3: Handle the remaining attributes with the Candidate Key

- **R3(A, B, C)** — Here, A is the Primary Key.

Final 3NF Decomposition:

1. **R1(D, E)**
2. **R2(B, C, D)**
3. **R3(A, B, C)**

14. a What is a conflict serializable schedule? Explain the precedence graph algorithm for testing conflict Serializability.

Determine whether the following schedule is conflict serializable. If yes, find an equivalent serial schedule:

$r1(X); r2(X); w1(X); r1(Y); w2(X); w1(Y)$

1. What is a Conflict Serializable Schedule?

- A schedule is conflict serializable if it is conflict equivalent to some serial schedule. In simpler terms, if we can transform a non-serial schedule into a serial one by swapping non-conflicting, adjacent operations, it is considered conflict serializable.
- Two operations are said to conflict if they belong to different transactions, access the same data item, and at least one of them is a write (w) operation.

The three types of conflicts are:

- Read-Write (RW) Conflict: $r_i(X), w_j(X)$
- Write-Read (WR) Conflict: $w_i(X), r_j(X)$
- Write-Write (WW) Conflict: $w_i(X), w_j(X)$

2. The Precedence Graph Algorithm

The Precedence Graph (or Serialization Graph) is a directed graph used to test for conflict Serializability.

The Algorithm Steps:

1. **Nodes:** Create a node for each transaction ($T_1, T_2, \dots, T_n$) appearing in the schedule.
2. **Edges:** Draw a directed edge from $T_i \rightarrow T_j$ if there is a conflict between an operation in T_i and a later operation in T_j . This happens if:
 - T_i reads/writes X before T_j writes X .
 - T_i writes X before T_j reads X .
3. **Test:** * If the graph contains a **cycle**, the schedule is **NOT** conflict serializable.
 - If the graph is **acyclic** (no cycles), the schedule **IS** conflict serializable.

3. Testing the Given Schedule

Schedule (S): $r1(X); r2(X); w1(X); r1(Y); w2(X); w1(Y)$

Let's identify the conflicts to build the graph:

Operation	Later Conflicting Operation	Dependency
$r2(X)$	$w1(X)$	$T2 \rightarrow T1$ (Read-Write conflict)
$w1(X)$	$w2(X)$	$T1 \rightarrow T2$ (Write-Write conflict)

Analysis:

- Because T2 reads X before T1 writes it, we have an edge **T2 → T1**.
- Because T1 writes X before T2 writes it, we have an edge **T1 → T2**.

Conclusion:

The resulting graph has edges T2→T1 and T1 →T2. This creates a **cycle** (T1 →T2).

Is it conflict serializable?

No, this schedule is **not conflict serializable** because the precedence graph contains a cycle.
Therefore, no equivalent serial schedule exists.

14.b Compare the deferred update and immediate update database recovery protocols. Explain shadow paging as an alternative recovery technique and state when it is preferred.

Feature	Deferred Update (No-Undo/Redo)	Immediate Update (Undo/Redo)
When Disk is Updated	Only after the transaction reaches its commit point .	During transaction execution (before commit).
Log Requirements	Records the new value (Redo log).	Records both old and new values (Undo and Redo logs).
Recovery Process	Only Redo operations are needed for committed transactions.	May require Undo for uncommitted and Redo for committed transactions.
Crash Handling	Transactions that were active at the time of the crash are simply ignored.	Transactions that were active must be rolled back using the "old values" in the log.
Efficiency	Lower overhead during execution but high memory usage for large transactions.	Higher overhead during execution due to frequent disk I/O.

2. Shadow Paging

Shadow Paging is a non-log-based recovery technique. It treats the database as a set of fixed-size disk pages and maintains two page tables during a transaction:

1. **Current Page Table:** Points to the most recent database pages, including those modified by the current transaction.
2. **Shadow Page Table:** A copy of the "Current" table made *before* the transaction starts. It points to the original, unmodified pages on the disk.

How it works:

- When a transaction writes to a page, a **new** page is allocated on the disk. The Current Page Table is updated to point to this new disk address, while the Shadow Page Table remains pointing to the old one.
- **On Commit:** The Shadow Page Table is discarded, and the Current Page Table becomes the permanent one.
- **On Failure:** The Current Page Table is discarded, and the system reverts to the Shadow Page Table, instantly restoring the database to its pre-transaction state.

3. When is Shadow Paging Preferred?

Shadow paging is generally preferred over log-based recovery in specific scenarios:

- **Small Databases/Simplicity:** It eliminates the overhead of maintaining and searching complex log files.
- **Minimal Recovery Time:** Recovery is almost instantaneous because you only need to flip the pointer back to the shadow page table; there is no need to "undo" or "redo" thousands of log records.
- **Where "Undo" is Costly:** In environments where logging every single change creates a massive performance bottleneck, shadow paging can be more efficient for atomic updates.

Note: The main disadvantage is "data fragmentation." Since new pages are written wherever space is available, the physical layout of the database can become disorganized over time.

Module 4

15. a Explain the BASE properties of NoSQL systems and compare them with ACID properties of relational databases. Use a real-world example (e.g., social media feed vs. banking transaction) to illustrate the trade-offs.

BASE Properties (NoSQL Systems)

NoSQL databases (like Cassandra or MongoDB) often follow the BASE model to handle massive amounts of data across many servers.

- **Basically Available:** The system guarantees availability; even if some nodes fail, the system stays up (though some data might be temporarily unreachable).
- **Soft State:** The state of the system may change over time, even without input, due to data being synchronized across different nodes.
- **Eventual Consistency:** The system will eventually become consistent. If no new updates are made to a data item, eventually all accesses to that item will return the last updated value.

Comparison of ACID vs. BASE

Feature	ACID (Relational Databases)	BASE (NoSQL Databases)
Philosophy	Strict consistency and reliability for critical data.	High availability, scalability, and flexibility, often at the cost of immediate consistency.
Consistency	Strong (Immediate) Consistency. Data is always up-to-date and correct across all replicas.	Eventual Consistency. Data might be inconsistent for a period after updates.
Availability	Can sometimes sacrifice availability for consistency (e.g., if a network partition occurs, some operations might be blocked).	Always tries to respond to requests, even if the data is stale or inconsistent.
Scalability	Traditionally scales vertically (more powerful server). Horizontal scaling (sharding) is possible but complex.	Designed for horizontal scaling (distributing data across many commodity servers).
Data Model	Structured, tabular (rows and columns), predefined schema.	Flexible, schema-less (key-value, document, graph, column-family).
Complexity	Easier to manage transactions, complex queries with JOINS.	Transactions often managed at the application level; JOINS typically not supported natively.

Feature	ACID (Relational Databases)	BASE (NoSQL Databases)
Use Cases	Financial transactions, e-commerce orders, inventory management.	Social media, real-time analytics, content delivery, IoT data.

Real-World Comparison: Banking vs. Social Media

The choice between ACID and BASE usually comes down to what "failure" looks like for your specific application.

The Banking Transaction (ACID)

Scenario: You transfer \$100 from your Savings to your Checking account.

- **The Trade-off:** Accuracy is non-negotiable. If the system crashes halfway through, you cannot have \$100 disappear from Savings without appearing in Checking.
- **Requirement:** The system must be **Consistent**. If the database cannot guarantee the exact balance right now, it should reject the transaction rather than provide "eventual" money.

The Social Media Feed (BASE)

Scenario: You post a photo on Instagram or a status update on X.

- **The Trade-off:** Availability and speed are prioritized. If you post a photo, it doesn't matter if your friend in London sees it \$2\$ seconds later than your friend in Kochi.
- **Requirement:** The system must be **Available**. It is better for the app to show a "slightly old" feed to millions of users than to shut down the entire platform because one server in a data centre is struggling to sync a "Like" count.

15. b Compare the four types of NoSQL architectural patterns: Key-Value Stores, Document Stores, Column Family Stores, and Graph Stores. For each, describe the data model and provide one industry use case.

1. Key-Value Stores

This is the simplest form of NoSQL database. Every item is stored as an attribute name ("key") together with its value. The database has no knowledge of what is inside the value; it simply retrieves it by the key.

- **Data Model:** A hash table or dictionary where a unique key maps to a BLOB (Binary Large Object), string, or JSON.
- **Industry Use Case: Session Management & Caching.** Websites like **Amazon** use key-value stores (e.g., Redis or Riak) to store user shopping carts or login sessions because retrieval is near-instantaneous complexity).

2. Document Stores

Document stores are an evolution of key-value stores. The "value" is a structured document (typically JSON, BSON, or XML) that the database can actually "read" and index.

- **Data Model:** A collection of independent documents. Unlike relational tables, every document in a collection can have a different structure (schema-less).
- **Industry Use Case: Content Management Systems (CMS).** Platforms like **Forbes** or **The New York Times** use document stores (e.g., MongoDB) to store articles. Since one article might have a video and another only text, the flexible schema is ideal.

3. Column-Family Stores (Wide-Column)

Instead of storing data in rows, these databases store data in column families. This allows for extremely fast queries across specific columns rather than reading entire rows of data.

- **Data Model:** A sparse, multi-dimensional sorted map. It consists of rows, and each row can have a massive, varying number of columns grouped into "families."
- **Industry Use Case: Time-Series Data & Analytics.** **Spotify** and **Facebook** use column-family stores (e.g., Apache Cassandra or HBase) to track user activity logs and real-time analytics across billions of data points.

4. Graph Stores

Graph databases focus on the **relationships** between data points. They treat the connections between data as being just as important as the data itself.

- **Data Model:** Consists of **Nodes** (entities like people or products), **Edges** (relationships like "friends with" or "purchased"), and **Properties** (details about nodes/edges).

- **Industry Use Case: Social Networks & Recommendation Engines.** LinkedIn uses graph stores (e.g., Neo4j) to map "1st, 2nd, and 3rd-degree" connections. It is also used in fraud detection to find suspicious patterns of links between accounts.

16. A social media company needs to manage the following data:

- **User profiles (name, bio, settings, profile picture URL)**
- **Posts (text, images, timestamps, likes count)**
- **Friend/follower connections and recommendations**
- **Real-time activity feed and notifications**

(a) For each data requirement, identify the most suitable type of NoSQL database and justify your choice. (5 marks)

(b) Design the data structure/schema for any two of your chosen NoSQL databases, showing sample documents/records. (4 marks)

a) Identify the most suitable type of NoSQL database and justify your choice.

Here's an analysis for each data requirement:

1. User profiles (name, bio, settings, profile picture URL)

➤ **Most Suitable NoSQL Type: Document Store**

Justification:

- User profiles are naturally represented as single, self-contained units of information.
- They often have a flexible schema (e.g., some users might have more detailed bios or different settings than others), which fits perfectly with the document model (e.g., JSON documents).
- A document store allows for easy retrieval of an entire profile based on a user ID, and complex profile objects can be stored efficiently without needing to join multiple tables.

2. Posts (text, images, timestamps, likes count)

➤ **Most Suitable NoSQL Type: Document Store or Column Family Store**

Justification:

- **Document Store (Primary choice):** Each post is a distinct entity with varying content (text, image URL, metadata).

- A document store allows storing a rich, hierarchical structure for each post (e.g., embedding comments or tags directly, or referencing them) and querying posts based on content or metadata like timestamps.
- **Column Family Store (Alternative for very high volume/sparse data):** If posts were extremely numerous and had potentially many varying, sparse attributes (e.g., different types of rich media metadata), a Column Family store could work well for its ability to handle wide rows with dynamic columns.
- However, for typical social media posts, the self-contained nature of a document store is usually a better fit.

3.Friend/follower connections and recommendations

➤ Most Suitable NoSQL Type: Graph Store

Justification:

- This requirement is a classic use case for graph databases. Relationships (friend, follower, recommended) are fundamental. Graph stores excel at efficiently storing and traversing complex, interconnected data structures.
- Operations like finding "friends of friends," "mutual followers," or generating recommendations based on network proximity are highly performant in a graph database, as relationships are first-class citizens.

4.Real-time activity feed and notifications

➤ Most Suitable NoSQL Type: Key-Value Store or Column Family Store

Justification:

- **Key-Value Store (Primary choice):** For an activity feed, the goal is often extremely fast reads and writes of simple items.
- A user's activity feed can be viewed as a list of items associated with a user ID (key). Key-value stores like Redis are perfect for quickly adding new activity items to a user's feed and retrieving the latest N items for display.
- They are often used for caching and leader boards, which have similar read/write patterns.
- **Column Family Store (Alternative for massive scale and personalized feeds):** For very large-scale, personalized feeds (e.g., Instagram, Twitter feeds), a Column Family store like Cassandra can be powerful.

- Each user's inbox feed could be a "row," and each activity item or post becomes a "column" with a timestamp key. This allows for very efficient retrieval of a user's personalized, sorted feed.

(b) Design the data structure/schema for any two of your chosen NoSQL databases.

Design for User Profiles (Document Store) and Friend/Follower Connections (Graph Store).

1. User Profiles (Document Store - JSON)

In a Document store, we can nest the user's settings directly within the profile document to avoid multiple lookups.

Collection: users

```
{
  "user_id": "u101",
  "name": "Arjun Das",
  "bio": "Tech enthusiast and traveler. Exploring the world one byte at a time.",
  "profile_pic_url": "https://cdn.social.com/pics/u101.jpg",
  "settings": {
    "private_account": false,
    "theme": "dark",
    "notifications_enabled": true
  },
  "created_at": "2026-01-15T10:30:00Z"
}
```

2. Friend Connections (Graph Database - Nodes & Edges)

In a Graph database, data is represented as entities (Nodes) and their relationships (Edges). This allows for easy recommendation logic (e.g., "People you may know").

Nodes (User):

- Node 1: {id: "u101", name: "Arjun"}

- Node 2: {id: "u102", name: "Sara"}
- Node 3: {id: "u103", name: "Sneha"}

Edges (Relationships):

- (Arjun)- [: FOLLOWS {since: "2026-02-01"}]->(Sara)
- (Sara)- [: FOLLOWS]->(Sneha)
- **(Recommendation Logic):** Since Arjun follows Sara, and Sara follows Sneha, the system can recommend Sneha to Arjun as a "Friend of a Friend."



2024 SCHEME

KTUNOTES.LIVE

SCAN THE QR CODES

join our WhatsApp group



For WhatsApp group

get into our website



website for more materials

CLICK THE LOGO TO JOIN



OUR TELEGRAM CHANNEL

for notes and discussion and updates



OUR WHATSAPP GROUP

for note updates and notifications



OUR WEBSITE

get in and download all 2024 materials